Project #1 Simulating Job Scheduler and Performance Analysis

Project #1 Description:

Simulating Job Scheduler of the Operating Systems by programming the following four scheduling algorithms that we covered in the class:

a. First-Come-First-Serve (FCFS)
b. Shortest-Job-First (SJF)
c. Round-Robin with Time Slice = 2 (RR-2)
d. Round-Robin with Time Slice = 5 (PR-5)

You can use either Java or your choice of programming language for the implementation. The objective of this project is to help student understand how above four job scheduling algorithms operates by implementing the algorithms, and conducting a performance analysis of them based on the performance measure of their average turnaround times (of all jobs) for each scheduling algorithm using multiple inputs. Output the details of each algorithm's execution. You need to show which jobs are selected at what times as well as their starting and stopping burst values. You can choose your display format, for examples, you can display the results of each in *Schedule Table* or *Gantt Chart* format (as shown in the class notes). The project will be divided into three parts (phases) to help you to accomplish above tasks in in a systematic and scientific fashion: Design and Testing, Implementation, and Performance Analysis.

The program will read process burst times from a file (job.txt) – this file will be generated by you. Note that you need to generate multiple testing cases (with inputs of 5 jobs, 10 jobs and 15 jobs). A sample input file of five jobs is given as follows (burst time in ms):

[Begin of job.txt]
Job1
7
Job2
18
Job3
10
Job4
4
Job5
12
[End of job.txt]

Note: you can assume that

(1) There are no more than 30 jobs in the input file (job.txt).
(2) Processes arrive in the order they are read from the file for FCFS, RR-2 and RR-5.
(3) All jobs arrive at time 0.
(4) FCFS uses the order of the jobs, Job1, Job2, Job3, …

You can implement the algorithms in your choice of data structures based on the program language of your choice. Note that you always try your best to give the most efficient program for each problem. The size of the input will be limited to be within 30 jobs.

Submission Instructions:

- ***turn in the following @Canvas after the completion of all three parts, part 1, part 2 and part 3***
  (1) four program files (your choice of programming language with proper documentation)
  (2) this document (complete all the answers)

Part1
Design & Testing (30 points)

a. Design the program by providing pseudocode or flowchart for each CPU scheduling algorithm.

For all algorithms: read job.txt into arrays name[0..n-1], burst[0..n-1], turnaround[0..n-1]

FCFS:

```
time = 0; total = 0;
FOR i = 0 TO n-1:
    start = time
    time = time + burst[i]
    turnaround[i] = time
    PRINT name[i], start, time
    total = total + turnaround[i]
PRINT total / n
```

SJF:

```
order = indices 0..n-1 sorted by burst ascending
time = 0; total = 0;
FOR each i IN order:
    start = time
    time = time + burst[i]
    turnaround[i] = time
    PRINT name[i], start, time
    total = total + turnaround[i]
PRINT total / n
```

Round-Robin (Q = 2 and Q = 5)

```
remaining[i] = burst[i] for all i
queue = [0, 1, ..., n-1]
time = 0; total = 0;
WHILE queue is not empty:
        i = DEQUEUE(queue)
        run = MIN(Q, remaining[i])
        start = time
        time = time + run
        remaining[i] = remaining[i] - run
        PRINT name[i], start, time, remaining[i]
        IF remaining[i] > 0: ENQUEUE(queue, i)
        ELSE:
                turnaround[i] = time
                total = total + time
PRINT total / n
```

b. Design the program correctness testing cases. Give at least 3 testing cases to test your program, and give the expected correct **average turnaround time** (for each testing case) in order to test the correctness of each algorithm.

Testing case #	Input (table of jobs with its job# and length	Expected output for FCFS (√ if Correct after testing in Part 2)	Expected output for SJF (√ if Correct after testing in Part 2)	Expected output for RR-2 (√ if Correct after testing in Part 2)	Expected output for RR-5 (√ if Correct after testing in Part 2)
1 (5 jobs)	Job 1: 7 Job 2: 18 Job 3: 10 Job 4: 4 Job 5: 12	ATT = 31.4 √	ATT = 24.0 √	ATT = 36.4 √	ATT = 36.0 √
2 (10 jobs)	Job 1: 6 Job 2: 3 Job 3: 9 Job 4: 5 Job 5: 12 Job 6: 2 Job 7: 8 Job 8: 4 Job 9: 10 Job 10: 1	ATT = 34.1 √	ATT = 23.1 √	ATT = 38.3 √	ATT = 38.2 √
3 (15 jobs)	Job 1: 8 Job 2: 4 Job 3: 15 Job 4: 6 Job 5: 11 Job 6: 3 Job 7: 9 Job 8: 14 Job 9: 2 Job 10: 7 Job 11: 12 Job 12: 5 Job 13: 10	ATT = 64.53 √	ATT = 45.33 √	ATT = 79.27 √	ATT = 78.47 √

	Job 14: 1 Job 15: 13				

c. Design testing strategy for the programs. Discuss about how to generate and structure the randomly generated inputs for experimental study later in Part 3.

Hint 1: To study the performance evaluation of the four job scheduling algorithms, this project will use three different input sizes, 5 jobs, 10 jobs and 15 jobs. It is the easiest to use a random number generator for generating the inputs. Note that you need to decide the maximum value of job length (use at least 20). However, student should store each data set in various sizes and use the same data set for each job scheduling algorithm.

The performance of average Turnaround Time of each input data size (5 jobs, 10 jobs and 15 jobs) can be calculated after an experiment is conducted in 20 trail (with 20 input sets of jobs). We can denote the results as the set X which contains the 20 computed Turnaround Times of 20 trails, where X = {x_1, x_2, x_3 ... x_{20}}, from the simulator.

For each data size (5 jobs, 10 jobs and 15 jobs):

$$\textit{Average Turnaround Time} = \frac{\sum_{i=1}^{20} Xi}{20}$$

The student should decide the maximum value of the job length (at least 20).

Correctness testing

- We can run each program on the three cases above and compare with the table.
- Add edge cases like 1 job with all equal bursts where SJF should equal FCFS, jobs shorter than the time quantum, and already-sorted ascending input where FCFS should equal SJF.

Random input generation for Part 3

- Job lengths are uniform random integers in between 1 and 30 which should be large enough including the round-robin preemption.
- n is either 5, 10, or 15 with 20 trials each, so 60 input files.
- Write each set to its own file in the job.txt format, like data/n05_t01.txt or data/n15_t20.txt. A fixed seed makes the experiment reproducible.
- It can generate once then feed the same 60 files to all four programs so differences come from the algorithm and not the data for consistency.
- For each file, record the average turnaround time. For each algorithm and size, compute the average of the 20 values for Part 3 table.

Part 2
Implementation (30 points)

a. Code each program based on the design (pseudocode or flow chart) in Part 1(a).

CHECK PROGRAM FILES FOR FULFILLING THIS PART

b. Document the program appropriately.

CHECK PROGRAM FILES FOR FULFILLING THIS PART

c. Test you program using the designed testing input data given in the table in Part 1(b), Make sure each program generates the correct answer by marking a "√" if it is correct for each testing case for each program column in the table. Repeat the process of debugging if necessary.

CHECK PROGRAM FILES and PART 1b FOR FULFILLING THIS PART

d. For each program, capture a screen shot of the execution (Compile&Run) using the testing case in Part 1(b) to show how this program works properly

FCFS:

```
$ python fcfs.py job5.txt
=== FCFS ===
Job       Start   Stop
Job1          0      7
Job2          7     25
Job3         25     35
Job4         35     39
Job5         39     51

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 |
0      7      25     35     39     51

Turnaround times:
  Job1    7
  Job2    25
  Job3    35
  Job4    39
  Job5    51

Average turnaround time = 31.40 ms
```

```
$ python fcfs.py job10.txt
=== FCFS ===
Job       Start   Stop
Job1          0      6
Job2          6      9
Job3          9     18
Job4         18     23
Job5         23     35
Job6         35     37
Job7         37     45
Job8         45     49
Job9         49     59
Job10        59     60

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 |
0      6      9      18     23     35     37     45     49     59     60

Turnaround times:
  Job1    6
  Job2    9
  Job3    18
  Job4    23
  Job5    35
  Job6    37
  Job7    45
  Job8    49
  Job9    59
  Job10   60

Average turnaround time = 34.10 ms
```

```
$ python fcfs.py job15.txt
=== FCFS ===
Job       Start   Stop
Job1          0      8
Job2          8     12
Job3         12     27
Job4         27     33
Job5         33     44
Job6         44     47
Job7         47     56
Job8         56     70
Job9         70     72
Job10        72     79
Job11        79     91
Job12        91     96
Job13        96    106
Job14       106    107
Job15       107    120

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 | Job11 | Job12 | Job13 |
0      8      12     27     33     44     47     56     70     72     79     91     96     106
| Job14 | Job15 |
106    107    120

Turnaround times:
  Job1    8
  Job2    12
  Job3    27
  Job4    33
```

```
  Job5    44
  Job6    47
  Job7    56
  Job8    70
  Job9    72
  Job10   79
  Job11   91
  Job12   96
  Job13   106
  Job14   107
  Job15   120

Average turnaround time = 64.53 ms
```

SJF:

```
$ python sjf.py job5.txt
=== SJF ===
Job       Start   Stop
Job4          0      4
Job1          4     11
Job3         11     21
Job5         21     33
Job2         33     51

Gantt chart:
| Job4 | Job1 | Job3 | Job5 | Job2 |
0      4      11     21     33     51

Turnaround times:
  Job1    11
  Job2    51
  Job3    21
  Job4    4
  Job5    33

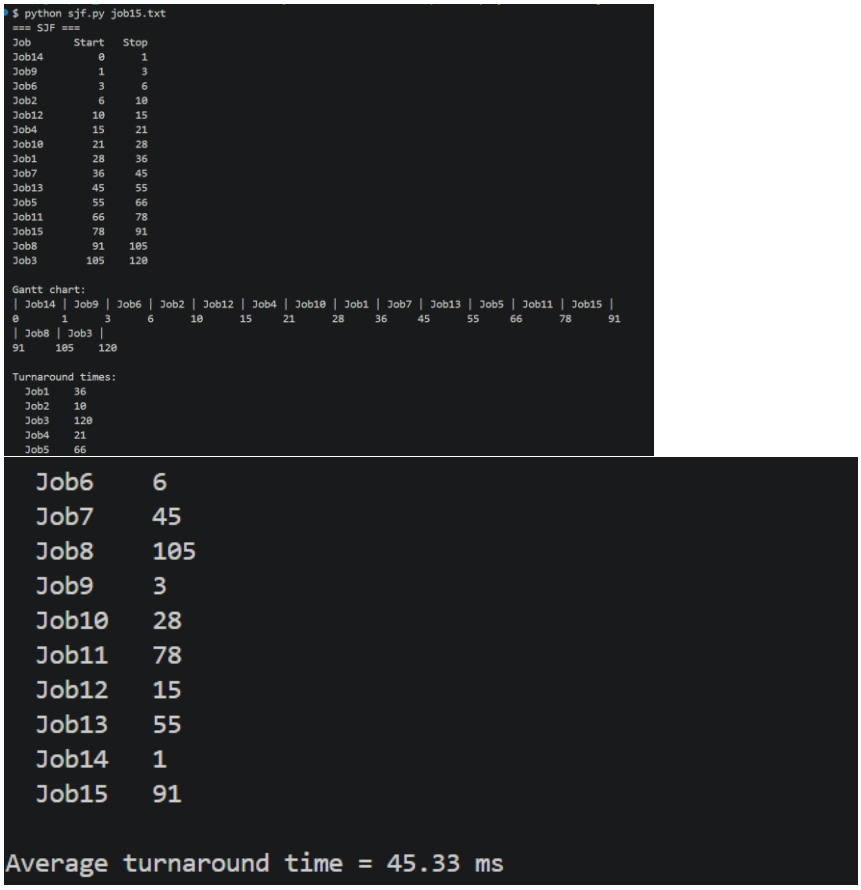
Average turnaround time = 24.00 ms
```

```
$ python sjf.py job10.txt
=== SJF ===
Job       Start   Stop
Job10         0      1
Job6          1      3
Job2          3      6
Job8          6     10
Job4         10     15
Job1         15     21
Job7         21     29
Job3         29     38
Job9         38     48
Job5         48     60

Gantt chart:
| Job10 | Job6 | Job2 | Job8 | Job4 | Job1 | Job7 | Job3 | Job9 | Job5 |
0       1      3      6      10     15     21     29     38     48     60

Turnaround times:
  Job1    21
  Job2    6
  Job3    38
  Job4    15
  Job5    60
  Job6    3
  Job7    29
  Job8    10
  Job9    48
  Job10   1

Average turnaround time = 23.10 ms
```


$ python sjf.py job15.txt
=== SJF ===
Job Start Stop
Job14 0 1
Job9 1 3
Job6 3 6
Job2 6 10
Job12 10 15
Job4 15 21
Job10 21 28
Job1 28 36
Job7 36 45
Job13 45 55
Job5 55 66
Job11 66 78
Job15 78 91
Job8 91 105
Job3 105 120
Gantt chart:
| Job14 | Job9 | Job6 | Job2 | Job12 | Job4 | Job10 | Job1 | Job7 | Job13 | Job5 | Job11 | Job15 |
0 1 3 6 10 15 21 28 36 45 55 66 78 91
| Job8 | Job3 |
91 105 120
Turnaround times:
Job1 36
Job2 10
Job3 120
Job4 21
Job5 66
Job6 6
Job7 45
Job8 105
Job9 3
Job10 28
Job11 78
Job12 15
Job13 55
Job14 1
Job15 91
Average turnaround time = 45.33 ms

RR2:

```
$ python rr2.py job5.txt
=== Round-Robin (time quantum = 2) ===
Job       Start   Stop  Burst before  Burst after
Job1          0      2             7            5
Job2          2      4            18           16
Job3          4      6            10            8
Job4          6      8             4            2
Job5          8     10            12           10
Job1         10     12             5            3
Job2         12     14            16           14
Job3         14     16             8            6
Job4         16     18             2            0
Job5         18     20            10            8
Job1         20     22             3            1
Job2         22     24            14           12
Job3         24     26             6            4
Job5         26     28             8            6
Job1         28     29             1            0
Job2         29     31            12           10
Job3         31     33             4            2
Job5         33     35             6            4
Job2         35     37            10            8
Job3         37     39             2            0
Job5         39     41             4            2
Job2         41     43             8            6
Job5         43     45             2            0
Job2         45     47             6            4
Job2         47     49             4            2
Job2         49     51             2            0

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job1 | Job2 | Job3 | Job4 | Job5 | Job1 | Job2 | Job3 | Job5 |
0      2      4      6      8      10     12     14     16     18     20     22     24     26     28
| Job1 | Job2 | Job3 | Job5 | Job2 | Job3 | Job5 | Job2 | Job5 | Job2 | Job2 | Job2 |
28     29     31     33     35     37     39     41     43     45     47     49     51

Turnaround times:
  Job1    29
  Job2    51
  Job3    39
  Job4    18
  Job5    45

Average turnaround time = 36.40 ms
```

```
$ python rr2.py job10.txt
=== Round-Robin (time quantum = 2) ===
Job       Start   Stop  Burst before  Burst after
Job1          0      2             6            4
Job2          2      4             3            1
Job3          4      6             9            7
Job4          6      8             5            3
Job5          8     10            12           10
Job6         10     12             2            0
Job7         12     14             8            6
Job8         14     16             4            2
Job9         16     18            10            8
Job10        18     19             1            0
Job1         19     21             4            2
Job2         21     22             1            0
Job3         22     24             7            5
Job4         24     26             3            1
Job5         26     28            10            8
Job7         28     30             6            4
Job8         30     32             2            0
Job9         32     34             8            6
Job1         34     36             2            0
Job3         36     38             5            3
Job4         38     39             1            0
Job5         39     41             8            6
Job7         41     43             4            2
Job9         43     45             6            4
Job3         45     47             3            1
Job5         47     49             6            4
Job7         49     51             2            0
Job9         51     53             4            2
Job3         53     54             1            0
Job5         54     56             4            2
Job9         56     58             2            0
Job5         58     60             2            0

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 | Job1 | Job2 | Job3 | Job4 |
0      2      4      6      8      10     12     14     16     18      19     21     22     24     26
| Job5 | Job7 | Job8 | Job9 | Job1 | Job3 | Job4 | Job5 | Job7 | Job9 | Job3 | Job5 | Job7 | Job9 |
26     28     30     32     34     36     38     39     41     43     45     47     49     51     53
| Job3 | Job5 | Job9 | Job5 |
53     54     56     58     60

Turnaround times:
  Job1    36
  Job2    22
  Job3    54
  Job4    39
  Job5    60
  Job6    12
  Job7    51
  Job8    32
  Job9    58
  Job10   19

Average turnaround time = 38.30 ms
```

```
$ python rr2.py job15.txt
=== Round-Robin (time quantum = 2) ===
Job        Start   Stop  Burst before  Burst after
Job1           0      2             8            6
Job2           2      4             4            2
Job3           4      6            15           13
Job4           6      8             6            4
Job5           8     10            11            9
Job6          10     12             3            1
Job7          12     14             9            7
Job8          14     16            14           12
Job9          16     18             2            0
Job10         18     20             7            5
Job11         20     22            12           10
Job12         22     24             5            3
Job13         24     26            10            8
Job14         26     27             1            0
Job15         27     29            13           11
Job1          29     31             6            4
Job2          31     33             2            0
Job3          33     35            13           11
Job4          35     37             4            2
Job5          37     39             9            7
Job6          39     40             1            0
Job7          40     42             7            5
Job8          42     44            12           10
Job10         44     46             5            3
Job11         46     48            10            8
Job12         48     50             3            1
Job13         50     52             8            6
Job15         52     54            11            9
Job1          54     56             4            2
Job3          56     58            11            9
Job4          58     60             2            0
Job5          60     62             7            5
Job7          62     64             5            3
Job8          64     66            10            8
Job10         66     68             3            1
Job11         68     70             8            6
Job12         70     71             1            0
Job13         71     73             6            4
Job15         73     75             9            7
Job1          75     77             2            0
Job3          77     79             9            7
```

```
Job5         79     81            5           3
Job7         81     83            3           1
Job8         83     85            8           6
Job10        85     86            1           0
Job11        86     88            6           4
Job13        88     90            4           2
Job15        90     92            7           5
Job3         92     94            7           5
Job5         94     96            3           1
Job7         96     97            1           0
Job8         97     99            6           4
Job11        99    101            4           2
Job13       101    103            2           0
Job15       103    105            5           3
Job3        105    107            5           3
Job5        107    108            1           0
Job8        108    110            4           2
Job11       110    112            2           0
Job15       112    114            3           1
Job3        114    116            3           1
Job8        116    118            2           0
Job15       118    119            1           0
Job3        119    120            1           0

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 | Job11 | Job12 | Job13 |
0      2      4      6      8      10     12     14     16     18      20      22      24      26
| Job14 | Job15 | Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job10 | Job11 | Job12 |
26      27      29     31     33     35     37     39     40     42     44      46      48      50
| Job13 | Job15 | Job1 | Job3 | Job4 | Job5 | Job7 | Job8 | Job10 | Job11 | Job12 | Job13 | Job15 |
50      52      54     56     58     60     62     64     66      68      70      71      73      75
| Job1 | Job3 | Job5 | Job7 | Job8 | Job10 | Job11 | Job13 | Job15 | Job3 | Job5 | Job7 | Job8 |
75     77     79     81     83     85      86      88      90      92     94     96     97     99
| Job11 | Job13 | Job15 | Job3 | Job5 | Job8 | Job11 | Job15 | Job3 | Job8 | Job15 | Job3 |
99      101     103     105    107    108    110     112     114    116    118     119    120

Turnaround times:
  Job1    77
  Job2    33
  Job3    120
  Job4    60
  Job5    108
  Job6    40
  Job7    97
  Job8    118
  Job9    18
  Job10   86
  Job11   112
  Job12   71
  Job13   103
  Job14   27
  Job15   119

 Average turnaround time = 79.27 ms
```

RR5:

```
$ python rr5.py job5.txt
 === Round-Robin (time quantum = 5) ===
 Job       Start   Stop  Burst before  Burst after
 Job1          0      5             7            2
 Job2          5     10            18           13
 Job3         10     15            10            5
 Job4         15     19             4            0
 Job5         19     24            12            7
 Job1         24     26             2            0
 Job2         26     31            13            8
 Job3         31     36             5            0
 Job5         36     41             7            2
 Job2         41     46             8            3
 Job5         46     48             2            0
 Job2         48     51             3            0

 Gantt chart:
 | Job1 | Job2 | Job3 | Job4 | Job5 | Job1 | Job2 | Job3 | Job5 | Job2 | Job5 | Job2 |
 0      5      10     15     19     24     26     31     36     41     46     48     51

 Turnaround times:
   Job1    26
   Job2    51
   Job3    36
   Job4    19
   Job5    48

 Average turnaround time = 36.00 ms
```

```
$ python rr5.py job10.txt
=== Round-Robin (time quantum = 5) ===
Job       Start   Stop  Burst before  Burst after
Job1          0      5             6            1
Job2          5      8             3            0
Job3          8     13             9            4
Job4         13     18             5            0
Job5         18     23            12            7
Job6         23     25             2            0
Job7         25     30             8            3
Job8         30     34             4            0
Job9         34     39            10            5
Job10        39     40             1            0
Job1         40     41             1            0
Job3         41     45             4            0
Job5         45     50             7            2
Job7         50     53             3            0
Job9         53     58             5            0
Job5         58     60             2            0

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 | Job1 | Job3 | Job5 | Job7 |
0      5      8      13     18     23     25     30     34     39      40     41     45     50     53
| Job9 | Job5 |
53     58     60

Turnaround times:
  Job1    41
  Job2    8
  Job3    45
  Job4    18
  Job5    60
  Job6    25
  Job7    53
  Job8    34
  Job9    58
  Job10   40

Average turnaround time = 38.20 ms
```

```
$ python rr5.py job15.txt
=== Round-Robin (time quantum = 5) ===
Job       Start   Stop  Burst before  Burst after
Job1          0      5             8            3
Job2          5      9             4            0
Job3          9     14            15           10
Job4         14     19             6            1
Job5         19     24            11            6
Job6         24     27             3            0
Job7         27     32             9            4
Job8         32     37            14            9
Job9         37     39             2            0
Job10        39     44             7            2
Job11        44     49            12            7
Job12        49     54             5            0
Job13        54     59            10            5
Job14        59     60             1            0
Job15        60     65            13            8
Job1         65     68             3            0
Job3         68     73            10            5
Job4         73     74             1            0
Job5         74     79             6            1
Job7         79     83             4            0
Job8         83     88             9            4
Job10        88     90             2            0
Job11        90     95             7            2
Job13        95    100             5            0
Job15       100    105             8            3
Job3        105    110             5            0
Job5        110    111             1            0
Job8        111    115             4            0
Job11       115    117             2            0
Job15       117    120             3            0

Gantt chart:
| Job1 | Job2 | Job3 | Job4 | Job5 | Job6 | Job7 | Job8 | Job9 | Job10 | Job11 | Job12 | Job13 |
0      5      9      14     19     24     27     32     37     39      44      49      54      59
| Job14 | Job15 | Job1 | Job3 | Job4 | Job5 | Job7 | Job8 | Job10 | Job11 | Job13 | Job15 | Job3 |
59      60      65     68     73     74     79     83     88      90      95      100     105    110
| Job5 | Job8 | Job11 | Job15 |
110    111    115     117     120

Turnaround times:
  Job1    68
  Job2    9
  Job3    110
  Job4    74
  Job5    111
  Job6    27
  Job7    83
  Job8    115
  Job9    39
  Job10   90
  Job11   117
  Job12   54
  Job13   100
  Job14   60
  Job15   120

Average turnaround time = 78.47 ms
```

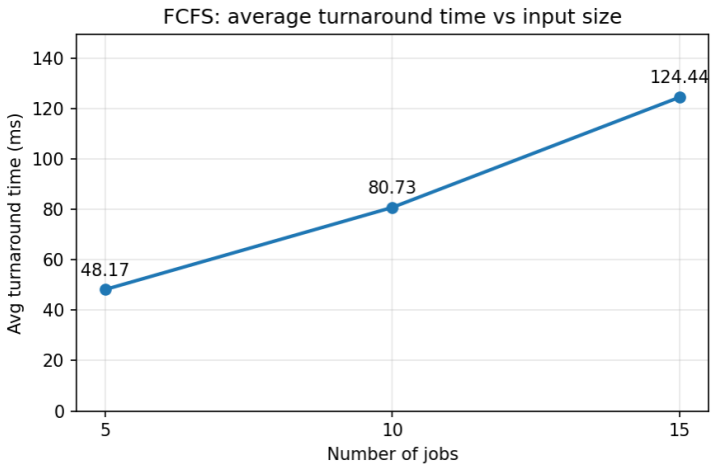
By now, four working programs are created and ready for experimental study in the next part, Part 3.

Part 3
Performance Analysis (40 points)

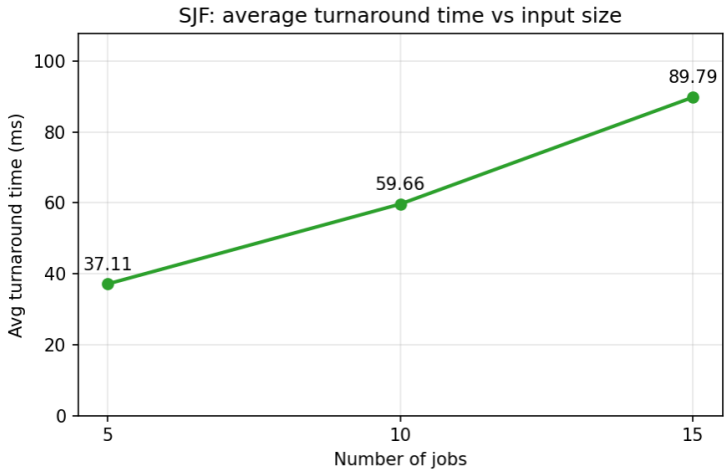
a. Run each program with the designed randomly generated input data given in Part 1(c). Generate a table for all the experimental results for performance analysis as follows.

Input Size n jobs	Average of average turnaround times (FCFS Program)	Average of average turnaround times (SFJ Program)	Average of average turnaround times (RR-2)	Average of average turnaround times (RR-5)
5 jobs	*AVG = 48.17*	*AVG = 37.11*	*AVG = 57.51*	*AVG = 57.63*
10 jobs	*AVG = 80.73*	*AVG = 59.65*	*AVG = 101.97*	*AVG = 101.97*
15 jobs	*AVG = 124.44*	*AVG = 89.79*	*AVG = 160.84*	*AVG = 158.91*

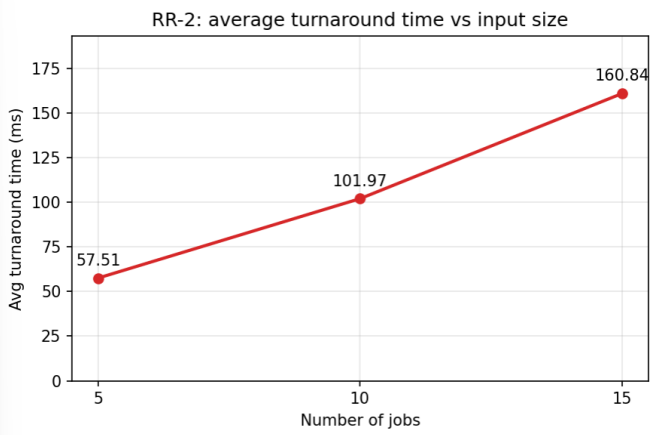
b. Plot a graph of each algorithm, average turnaround time vs input size (# of jobs), and summarize the performance of each algorithm based on its own graph.



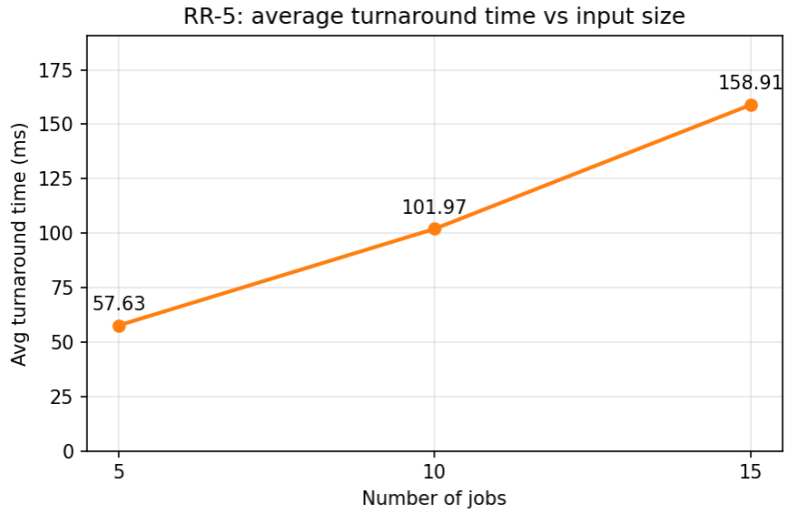
The average turnaround time rises from 48.17 at 5 jobs to 80.73 at 10 jobs and 124.44 at 15 jobs. Each added job waits for every job before it so the wait grows with the queue length. FCFS ignores job length, so a long job early in the file delays everything behind it.



SJF goes up from 37.11 to 59.65 to 89.79, which is the lowest at every size between all of the algorithms since running short jobs first puts many short completion times early, so only a few jobs wait behind the long ones.

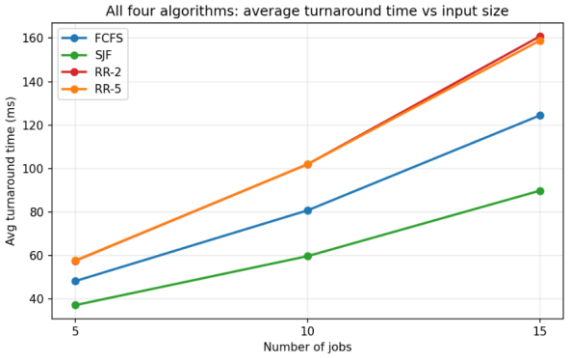


RR-2 rises from 57.51 to 101.97 to 160.84. Every job gets many short turns, so even short jobs wait through several rounds and long jobs don't finish until near the end. Since all jobs arrive at time 0, this pushes the average up.



RR-5 rises from 57.63 to 101.97 to 158.91 ms. It is pretty much the same as RR-2.

Plot all four graphs on the same graph and compare the performance of all four algorithms. Rank four scheduling algorithms. Try giving the reasons for the findings.



The round-robins rank below FCFS because they are built for responsiveness, giving every job an early first turn while turnaround time measures only when a job finishes. Slicing jobs delays every finish so FCFS had a lower average.
SJF's lead over FCFS grows with input size, from about 11 ms at 5 jobs to 21 ms at 10 jobs and 35 ms at 15 jobs because a bad ordering costs more as the queue gets longer.
RR-5 and RR-2 is a near tie. RR-5 comes out slightly ahead overall because a larger time slice lets jobs finish in fewer turns so it behaves a bit more like FCFS, and that effect shows up mainly at 15 jobs.

c. Conclude your report with the strength and constraints of your work. At least 100 words. (Note: It is reflection of this project. If you have a change to re-do this project again, what you like to keep and what you like to do differently in order get a better quality of results.)

Strengths:
The main strength of this project is that the comparison was fair. I generated the 60 input sets once with a fixed seed, saved each one in the same format as job.txt, and ran all four schedulers on those identical files, so differences in the results come from the algorithms only and not ambiguity in data. Before the experiment I checked each program against hand-computed answers from Part 1, and the results matched, which gave me confidence that the programs were correct. The experiment also confirmed what theory predicts, which is that SJF had the lowest average turnaround time at every size and did better than FCFS in all 20 trials.

Constraints:
All jobs arrive at time 0, so the results cannot show where round-robin is better such as response time or jobs that arrive over time. I ignored context-switch cost overhead, which would make RR-2 look worse than RR-5. I tested only one job-length distribution, being uniform from 1 to 30, three input sizes, and 20 trials each, which was not enough to separate RR-2 from RR-5 with confidence. If I did this again, I would keep the stored, shared input files and the fixed seed. I would add more trials, other job length distributions, a context-switch cost, and a response-time measure. I would also use a confidence interval or a paired statistical test to decide whether two close algorithms like RR2 and RR5 really differ (like I learned in stats class).